

ETHENA · SECURITY REVIEW

Ethena

Onchain Minting.

A security review of Onchain Minting.

CLIENT	PROJECT	AUDITORS	AUDIT WINDOW	FINAL REPORT
Ethena	Onchain Minting	Guardian	June 10 – June 25, 2026	July 2, 2026

§ 00

Contents.

33 pages — overview, confidence ranking, scope, 21 findings, disclaimer, and about Guardian.

§ 01	Overview	P. 03
§ 02	Confidence Ranking	P. 04
§ 03	Scope & Classifications	P. 05
	Files in Scope	P. 06
§ 04	Findings	P. 07
	Main Review · 9 findings	P. 09
	Remediation Review · 12 findings	P. 19
§ 05	Disclaimer	P. 32
§ 06	About Guardian	P. 33

§ 01

Overview.

Guardian reviewed Onchain Minting across 2 review rounds. The final report includes 3 Medium, 9 Low, 9 Informational findings.

AUDIT FIRM	METHODOLOGY
Guardian	Smart contract · Manual · Main Review · Remediation Review
CLIENT	
Ethena	
PROJECT	
Onchain Minting	
PREPARED BY	
Guardian	
AUDIT WINDOW	
June 10 – June 25, 2026	
FINAL REPORT	
July 2, 2026	

1.1 Confidence Ranking

5 / 5

Very High Confidence

Guardian assigns a Confidence Ranking of 5 out of 5, combining finding severity, remediation outcome, and the reviewer’s assessment of latent risk. The full scale is on the next page.

1	2	3	4	5
VERY LOW CONFIDENCE	LOW CONFIDENCE	MODERATE CONFIDENCE	HIGH CONFIDENCE	VERY HIGH CONFIDENCE

1.2 Findings at a Glance

SEVERITY	TOTAL	PENDING	ACK	PARTIAL	RESOLVED
Critical	0	0	0	0	0
High	0	0	0	0	0
Medium	3	0	3	0	0
Low	9	0	8	0	1
Informational	9	0	9	0	0
All severities	21	0	20	0	1

§ 02

Confidence Ranking.

Guardian assigns a project-level confidence score using the scale below. The ranking combines finding severity, remediation outcome, and the reviewer's assessment of latent risk.

CONFIDENCE RANKING	DEFINITION AND RECOMMENDATION	RISK PROFILE
5 Very High Confidence ASSIGNED	Codebase is mature, clean, and secure. No High or Critical vulnerabilities were found. Follows modern best practices with high test coverage and thoughtful design. Recommendation: code is highly secure at time of audit.	0 High/Critical findings and few Low/Medium severity findings.
4 High Confidence	Code is clean, well-structured, and adheres to best practices. Design patterns are sound and test coverage is strong. Recommendation: suitable for deployment after remediations; consider periodic review with changes.	0-1 High/Critical findings per engagement week and little to no Medium severity findings.
3 Moderate Confidence	Code is functional but shows gaps in defensive programming or test coverage. Recommendation: remediate the findings and re-review the changes before deployment.	Some High/Critical findings, with various Medium severity findings.
2 Low Confidence	Recurring issues suggest systemic gaps rather than isolated mistakes. Recommendation: remediate, then commission a follow-up review at a frozen commit.	Over 6 High and Critical findings across the engagement.
1 Very Low Confidence	The volume and severity of findings indicate the codebase is not ready for production. Recommendation: substantial rework, then a full re-review.	Over 12 High and Critical findings across the engagement.

§ 03

Scope & Classifications.

Guardian reviewed Onchain Minting.

3.1 Severity Matrix

	IMP · HIGH	IMP · MED	IMP · LOW
LIK · HIGH	Critical	High	Medium
LIK · MED	High	Medium	Low
LIK · LOW	Medium	Low	Low

IMPACT · HIGH

Significant loss of assets, harm to a group of users, or core protocol functionality is disrupted.

IMPACT · MEDIUM

Small amount of funds can be lost, or ancillary functionality is affected. Users may experience reduced or delayed receipt.

IMPACT · LOW

Unexpected behavior in some functionality that is notable but does not meet the criteria for higher severity.

LIKELIHOOD · HIGH

Possible with reasonable assumptions that mimic on-chain conditions; cost is low compared to amount gained.

LIKELIHOOD · MEDIUM

Possible only in uncommon cases or requires a large amount of capital relative to disruption caused.

LIKELIHOOD · LOW

Unlikely to ever occur in production.

Files in Scope.

FILE	NSLOC	TOTAL
No files were recorded in scope.		

§ 04

Findings.

Grouped by round. Visible IDs reset by severity within each round, so a finding is named the same way in the report as it was in the channel it was written in.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
MAIN REVIEW					
M-01	Token Blacklist Does Not Freeze PSM Routing	Validation	MEDIUM	ACKNOWLEDGED	p. 9
L-01	Self-transfer allows free swaps	Trust Assumptions	LOW	ACKNOWLEDGED	p. 11
L-02	Misleading collateral config update event	Events	LOW	ACKNOWLEDGED	p. 12
L-03	Self-approval getter mismatch	Unexpected Behavior	LOW	ACKNOWLEDGED	p. 13
I-01	Unknown collateral can be enabled	Unexpected Behavior	INFORMATIONAL	ACKNOWLEDGED	p. 14
I-02	Weak granularity in PSM events	Events	INFORMATIONAL	ACKNOWLEDGED	p. 15
I-03	Underdocumented delegated signer power	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 16
I-04	Zero Custom Fee Falls Back to Default	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 17
I-05	Quote calls pollute oracle events	Events	INFORMATIONAL	ACKNOWLEDGED	p. 18
REMEDIATION REVIEW					
M-01	Stale Peg Enables PSM Arbitrage	Gaming	MEDIUM	ACKNOWLEDGED	p. 19
M-02	Blacklist Redirect Queues OFT Compose	Logical Error	MEDIUM	ACKNOWLEDGED	p. 20
L-01	Fee Events Overstate Fees	Events	LOW	RESOLVED	p. 22

Findings continued.

ID	TITLE	CATEGORY	SEVERITY	STATUS	PAGE
L-02	Oracle Bounds Can Freeze PSM Swaps	DoS	LOW	ACKNOWLEDGED	p. 23
L-03	Old Oracle Timestamp Can Halt PSM Swaps	Oracle	LOW	ACKNOWLEDGED	p. 24
L-04	Re-Adding Benefactor Revives Old Authorizations	Warning	LOW	ACKNOWLEDGED	p. 25
L-05	Aggregate Oracle Counts Duplicate Sources	Oracle	LOW	ACKNOWLEDGED	p. 26
L-06	PSM Ignores Pyth Confidence Bounds	Oracle	LOW	ACKNOWLEDGED	p. 27
I-01	PSM Deploy Config Values Can Wrap	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 28
I-02	Stale Oracle Aggregation Docs	Documentation	INFORMATIONAL	ACKNOWLEDGED	p. 29
I-03	Inactive Benefactor Can Be Spent As Custodian	Informational	INFORMATIONAL	ACKNOWLEDGED	p. 30
I-04	Unsafe Downcast Before Quote Selection	Math	INFORMATIONAL	ACKNOWLEDGED	p. 31

MEDIUM **M-01**

ACKNOWLEDGED

Token Blacklist Does Not Freeze PSM Routing

CATEGORY Validation

LOCATION <https://github.com/GuardianOrg/onchain-minting-internal-team1-1781041543948/blob/fac5fee8f80693cff880c6f08dd3432aacfcbae/src/swap/PSM.sol#L266>

REVIEW Main Review

DESCRIPTION

xUSD/OFT blocks blacklisted operators from moving tokens through approvals by checking `msg.sender`, `from`, and `to` in `_update()`. This prevents a blacklisted spender from calling `transferFrom()` against approval granted by a clean address. The documentation states that the `msg.sender` blacklist check exists to prevent blacklisted addresses from circumventing restrictions through approvals.

PSM maintains separate routing authority through approved beneficiaries and delegated signers, but does not integrate that authority with the xUSD/OFT blacklist. A blacklisted active benefactor can still approve a clean beneficiary, and a blacklisted address can become, remain, or confirm as a delegated signer without any blacklist check.

When a PSM swap executes, the xUSD transfer is performed by the PSM. The token therefore sees `msg.sender == address(psm)` and only the transfer endpoints as `from` and `to`. If those endpoints are clean, the transfer succeeds even though the actor controlling the PSM routing path is blacklisted.

For `swapForAsset`, xUSD moves from the asset send custodian to the clean beneficiary, so a blacklisted benefactor or delegated signer can route custodian xUSD as long as the PSM-level beneficiary/signing checks pass. For `swapForCollateral`, a blacklisted delegated signer can initiate xUSD movement from a clean benefactor to the asset receive custodian.

As a result, token blacklisting alone does not freeze PSM-mediated routing authority. This undermines the documented `msg.sender` blacklist protection intended to stop blacklisted actors from moving tokens through clean accounts or approvals.

RECOMMENDATION

Add PSM-level blacklist checks at swap execution for blacklisted PSM actors that are not necessarily visible to xUSD/OFT during the token transfer. At minimum, `swap()` should reject when `msg.sender` is blacklisted, and should also reject `swapForAsset` when `order.benefactor` is blacklisted, because in that direction xUSD moves from the asset send custodian to the beneficiary and the token does not see the benefactor. Existing xUSD/OFT `from / to` checks

M-01 Token Blacklist Does Not Freeze PSM Routing

CONTINUED

already cover token-visible endpoints such as the beneficiary in `swapForAsset` and the benefactor in `swapForCollateral`.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

ACKNOWLEDGED

Self-transfer allows free swaps

CATEGORY	Trust Assumptions
LOCATION	https://github.com/GuardianOrg/onchain-minting-internal-team1-1781041543948/blob/fac5fee8f80693cff880c6f08dd3432aaacfcbae/src/swap/PSM.sol#L323-L331
REVIEW	Main Review

— DESCRIPTION

The PSM assumes every swap transfers input value from the benefactor into the input custodian before sending output value from the opposite custodian. This assumption breaks if the input custodian is also an active benefactor.

```
if (_isSwapForAsset) {  
    IERC20(order.collateral)  
        .safeTransferFrom(order.benefactor, _collateralConfig.receiveCustodianAddress,  
order.amountIn);  
    asset.safeTransferFrom(assetSendCustodianAddress, order.beneficiary, amountOut);  
} else {  
    asset.safeTransferFrom(order.benefactor, assetReceiveCustodianAddress, order.amountIn);  
    IERC20(order.collateral)  
        .safeTransferFrom(_collateralConfig.sendCustodianAddress, order.beneficiary, amountOut);  
}
```

When `order.benefactor` equals the collateral receive custodian, the first transfer is a self-transfer and does not increase custodian collateral balances, while the asset custodian still pays `amountOut`. The same issue exists for `swapForCollateral` when `order.benefactor` equals `assetReceiveCustodianAddress`.

If a custodian address is accidentally enabled as a benefactor, it can extract value from the opposite custodian without providing net input value.

— RECOMMENDATION

Prevent custodian addresses from being used as benefactors in the corresponding swap direction.

— RESOLUTION

Status: Acknowledged.

LOW **L-02**

ACKNOWLEDGED

Misleading collateral config update event

CATEGORY	Events
LOCATION	https://github.com/GuardianOrg/onchain-minting-internal-team1-1781041543948/blob/fac5fee8f80693cff880c6f08dd3432aaacfcbae/src/swap/PSM.sol#L550-L553
REVIEW	Main Review

DESCRIPTION

`updateCollateralConfig` preserves the existing active status in storage, but emits the raw input config instead of the final stored config. If `config.isActive` differs from `wasActive`, the emitted `CollateralConfigUpdated` event reports a state that was never stored. Consequently, off-chain indexers and monitoring systems can track an incorrect collateral state.

RECOMMENDATION

Emit the stored config instead of the calldata config, because storage already contains the normalized `isActive` value.

RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Self-approval getter mismatch

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/ethena-onchain-minting-source-team1-1780950361840/blob/main/src/swap/PSM.sol#L1143-L1145
REVIEW	Main Review

DESCRIPTION

The swap validation treats a benefactor as implicitly approved when the benefactor is also the beneficiary, but `isApprovedBeneficiary` only returns the explicit mapping value.

```
if (order.benefactor != order.beneficiary &&
    !_benefactorState.config.approvedBeneficiaries[order.beneficiary])
{
    revert BeneficiaryNotApproved(order.beneficiary);
}
```

As a result, the getter can return false even though the same benefactor-beneficiary pair is valid for swaps. Frontends or integrators relying on this getter may incorrectly block valid self-beneficiary swaps.

RECOMMENDATION

Consider updating `isApprovedBeneficiary` to return true when `benefactor == beneficiary`.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-01

ACKNOWLEDGED

Unknown collateral can be enabled

CATEGORY	Unexpected Behavior
LOCATION	https://github.com/GuardianOrg/onchain-minting-internal-team1-1781041543948/blob/fac5fee8f80693cff880c6f08dd3432aaacfcbae/src/swap/PSM.sol#L449-L461
REVIEW	Main Review

DESCRIPTION

`enableCollateral` can mark any non-zero address as active without verifying that the collateral was previously added or configured. Consequently, an unknown collateral can mistakenly end up with `isActive == true` while all other config fields remain zero, including the oracle and custodian addresses. Later, `swap` only checks the active flag before reading the oracle, so it will attempt to call `getPrice` on `address(0)` and revert instead of failing with the expected `CollateralNotSupported` error.

RECOMMENDATION

Consider requiring that the collateral already has a valid stored config before enabling it.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-02**

ACKNOWLEDGED

Weak granularity in PSM events

CATEGORY	Events
LOCATION	PSM.sol
REVIEW	Main Review

— DESCRIPTION

Many PSM configuration events do not include the previous value, and some emit only a generic marker for several different state changes. This affects collateral configuration, benefactor configuration, global/default limits, peg price updates, and lifecycle toggles.

As a result, off-chain monitors cannot reconstruct the exact state transition from logs alone. They must query storage after the transaction and may miss the old value, the specific changed field, or the precise semantics of the update.

— RECOMMENDATION

Consider emitting specific events that include both old and new values for every PSM configuration change.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-03**

ACKNOWLEDGED

Underdocumented delegated signer power

CATEGORY	Documentation
LOCATION	Global
REVIEW	Main Review

— DESCRIPTION

The PSM docs mention delegation only as part of benefactor validation and nonce sharing, but they do not clearly document that an accepted delegated signer can execute swaps with effectively the same swap authority as the benefactor.

Once accepted, a delegated signer can execute any valid swap for the benefactor, subject to the benefactor's approved beneficiaries, limits, balances, and token allowances. This includes consuming benefactor nonces and moving benefactor funds.

As a result, integrators or operators may underestimate the trust level required for delegated signers and treat them as lower-risk accounts than they actually are.

— RECOMMENDATION

Consider documenting that an accepted delegated signer has full swap execution authority for the benefactor within configured PSM constraints, and adding the same warning to the relevant NatSpec for delegation functions.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-04**

ACKNOWLEDGED

Zero Custom Fee Falls Back to Default

CATEGORY	Documentation
LOCATION	https://github.com/GuardianOrg/onchain-minting-internal-team1-1781041543948/blob/fac5fee8f80693cff880c6f08dd3432aaacfcbae/src/swap/PSM.sol#L1150
REVIEW	Main Review

— DESCRIPTION

The PSM implementation reserves a custom fee value of 0 as an unset sentinel. When no zero-fee exemption is set, a custom fee of 0 falls back to the collateral default fee rather than producing a zero-fee swap.

However, the docs state that custom fees can be set to zero for fee-free operations. This can cause operators or integrators to believe that calling `setBenefactorSwapForAssetFee(benefactor, collateral, 0)` or `setBenefactorSwapForCollateralFee(benefactor, collateral, 0)` grants a zero custom fee. In practice, zero-fee behavior requires either a zero collateral default fee or the explicit zero-fee exemption flag.

As a result, fee policy may be misunderstood or misconfigured by operators relying on the broader documentation instead of the implementation-specific NatSpec.

— RECOMMENDATION

Clarify the fee documentation to state that 0 custom fee means “unset / use collateral default”, and that fee-free per-benefactor swaps require the explicit zero-fee exemption flag unless the collateral default fee is also zero.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-05

ACKNOWLEDGED

Quote calls pollute oracle events

CATEGORY	Events
LOCATION	https://github.com/GuardianOrg/onchain-minting-internal-team1-1781041543948/blob/fac5fee8f80693cff880c6f08dd3432aaacfcbae/src/swap/PSM.sol#L1484
REVIEW	Main Review

— DESCRIPTION

`getQuote` is externally callable and routes through oracle validation before returning the quoted amounts. The `_validateOraclePrice` shared validation helper emits `OraclePriceValidated` whenever the oracle data passes validation.

As a result, anyone can generate oracle validation logs by calling `getQuote` without executing a proper swap. This can pollute monitoring and indexers that treat `OraclePriceValidated` as evidence of swap-time oracle validation.

— RECOMMENDATION

Consider splitting oracle validation into a non-emitting helper for `getQuote` and an emitting path for actual swaps only.

— RESOLUTION

Status: Acknowledged.

MEDIUM **M-01**

ACKNOWLEDGED

Stale Peg Enables PSM Arbitrage

CATEGORY Gaming

LOCATION <https://github.com/GuardianOrg/onchain-minting-internal-team1-1782152411467/blob/354bc9b8019586b8e1d75fc8c8998a8507b55a1b/src/swap/PSM.sol#L1571-L1631>

REVIEW Remediation Review

DESCRIPTION

The `swap` function prices the primary asset using the currently configured `pegPrice`. The collateral price is fetched from an oracle and validated during the swap, but the asset side relies on the last value set by the peg manager.

The protocol includes `setPegPrice`, which allows `PEG_MANAGER_ROLE` to update the asset peg when the asset permanently depegs to a new equilibrium.

However, this creates a race condition during asset repricing events. If the asset trades at \$0.80 while `pegPrice` is still \$1, an attacker can buy the asset externally and call `swapForCollateral` before the peg manager's update is executed, receiving close to \$1 of collateral for each discounted asset token. If the asset trades at \$1.20 while `pegPrice` is still \$1, an attacker can call `swapForAsset` before the update, sending \$1 of collateral and receiving asset worth about \$1.20.

The existence of `PEG_MANAGER_ROLE` mitigates long-term stale pricing, but it does not prevent short stale price windows. Malicious users can monitor the market and mempool and execute profitable swaps before the peg update is executed.

RECOMMENDATION

Add asset-side price protection so swaps revert when the asset market price diverges from the configured `pegPrice`.

RESOLUTION

Status: Acknowledged.

MEDIUM **M-02**

ACKNOWLEDGED

Blacklist Redirect Queues OFT Compose

CATEGORY	Logical Error
LOCATION	https://github.com/GuardianOrg/onchain-minting-internal-team1-1782152411467/blob/354bc9b8019586b8e1d75fc8c8998a8507b55a1b/src/oft/xUSD0FTUpgradeable.sol#L402-L416
REVIEW	Remediation Review

— DESCRIPTION

`xUSD0FTUpgradeable._credit` redirects a blacklisted destination's tokens to `rescueRecipient` but returns the positive amount credited there. The inherited OFT receive path then uses the original decoded `toAddress` for `sendCompose` and `OFTReceived` event, encoding that positive `amountReceivedLD` in the compose payload even though the original recipient received no xUSD.

M-02 Blacklist Redirect Queues OFT Compose

CONTINUED

```
address toAddress = _message.sendTo().bytes32ToAddress();
// @dev Credit the amountLD to the recipient and return the ACTUAL amount the recipient
received in local decimals
uint256 amountReceivedLD = _credit(toAddress, _toLD(_message.amountSD()), _origin.srcEid);
if (_message.isComposed()) {
// @dev Proprietary composeMsg format for the OFT.
bytes memory composeMsg = OFTComposeMsgCodec.encode(
    _origin.nonce,
    _origin.srcEid,
    amountReceivedLD,
    _message.composeMsg()
);
// @dev Stores the lzCompose payload that will be executed in a separate tx.
// Standardizes functionality for executing arbitrary contract invocation on some non-evm
chains.
// @dev The off-chain executor will listen and process the msg based on the src-chain-callers
compose options passed.
// @dev The index is used when a OApp needs to compose multiple msgs on lzReceive.
// For default OFT implementation there is only 1 compose msg per lzReceive, thus its always
0.
endpoint.sendCompose(toAddress, _guid, 0 /* the index of the composed message*/, composeMsg);
}
emit OFTReceived(_guid, _origin.srcEid, toAddress, amountReceivedLD);
```

Consequently, a blacklisted compose receiver can still receive an authenticated LayerZero compose callback from the xUSD OFT reporting a positive received amount while its xUSD balance remains zero. Compose-based integrations or accounting that trust the OFT callback as receipt proof can release or credit downstream assets despite blacklist redirection.

— RECOMMENDATION

When a blacklist redirect occurs, suppress compose/event attribution to the original recipient, redirect compose and events to `rescueRecipient`, or make the receive path encode/return zero for the blacklisted original recipient.

— RESOLUTION

Status: Acknowledged.

LOW **L-01**

RESOLVED

Fee Events Overstate Fees

CATEGORY	Events
LOCATION	https://github.com/GuardianOrg/onchain-minting-internal-team1-1782152411467/blob/354bc9b8019586b8e1d75fc8c8998a8507b55a1b/src/swap/PSM.sol#L1598-L1631
REVIEW	Remediation Review

DESCRIPTION

The PSM computes `feeAmount` before calculating the final swap output. The peg-based quote applies this fee by using `netAmountIn`, but the oracle-based quote uses the full gross `amountIn`. Since the final output is the minimum of the two quotes, the oracle-based quote can win even though it did not apply the computed fee.

When the peg-based path wins, the emitted `feeAmount` corresponds to the reduced output. When the oracle-based path wins, `SwapExecuted` may emit a non-zero `feeAmount` even though the user output was calculated from the gross input amount. This can happen during collateral price deviations, where the oracle path is selected to protect the protocol.

Consequently, off-chain accounting systems that rely on `SwapExecuted.feeAmount` can overstate actually collected fees.

RECOMMENDATION

Emit the effective collected fee instead of the precomputed fee. If the oracle path wins, set the emitted fee to 0, or split the event into `quotedFeeAmount` and `effectiveFeeAmount` so monitoring systems can distinguish configured fees from fees actually applied.

RESOLUTION

Status: Resolved.

LOW **L-02**

ACKNOWLEDGED

Oracle Bounds Can Freeze PSM Swaps

CATEGORY	DoS
LOCATION	https://github.com/GuardianOrg/onchain-minting-internal-team1-1782152411467/blob/354bc9b8019586b8e1d75fc8c8998a8507b55a1b/src/oracle/AggregateOracleFeed.sol#L235-L266
REVIEW	Remediation Review

— DESCRIPTION

The aggregate oracle applies its own minimum and maximum price bounds before returning a price to the PSM. These bounds are immutable, while the PSM collateral configuration can be updated by the collateral manager. As a result, if a collateral asset reaches a new valid market equilibrium outside the aggregate oracle's deployment-time bounds, the PSM may remain unable to swap even after its own collateral bounds are updated.

Because the PSM calls the configured oracle feed before applying its own collateral bounds, a revert inside the aggregate oracle prevents the updated PSM bounds from being evaluated.

— RECOMMENDATION

Consider making aggregate oracle bounds mutable through an admin-gated function, or ensure recovery procedures replace the collateral's oracle feed with a newly deployed feed using wider bounds.

— RESOLUTION

Status: Acknowledged.

LOW **L-03**

ACKNOWLEDGED

Old Oracle Timestamp Can Halt PSM Swaps

CATEGORY Oracle
LOCATION AggregateOracleFeed.sol
REVIEW Remediation Review

DESCRIPTION

AggregateOracleFeed filters oracle members using its own maxStaleness, calculates the median price from the remaining valid prices, and returns the earliest timestamp among those valid members.

PSM consumes that returned timestamp as the oracle updatedAt value and applies a separate collateral-level maxOracleAge check. If the aggregate oracle uses a wider staleness window than the PSM collateral config, one lagging oracle can make PSM reject swaps even when the aggregate median is valid and enough fresher oracle members exist.

For example, with AggregateOracleFeed.maxStaleness = 24 hours and PSM.CollateralConfig.maxOracleAge = 1 hour, two fresh feeds plus one 2-hour-old feed return a valid median price but an old aggregate timestamp. PSM then reverts with OraclePriceTooOld.

As a result, a single lagging-but-aggregate-valid oracle member can halt swaps for the affected collateral until the oracle updates, the member is removed, aggregate staleness is reduced, or PSM max age is widened.

RECOMMENDATION

Enforce freshness-window alignment for PSM collaterals using aggregate feeds. At minimum, ensure AggregateOracleFeed.maxStaleness <= CollateralConfig.maxOracleAge during deployment and config updates, or make the aggregate oracle apply the PSM freshness threshold directly before returning a timestamp.

RESOLUTION

Status: Acknowledged.

LOW **L-04**

ACKNOWLEDGED

Re-Adding Benefactor Revives Old Authorizations

CATEGORY	Warning
LOCATION	PSM.sol
REVIEW	Remediation Review

DESCRIPTION

`removeBenefactor()` deletes `benefactorState[benefactor].config`, but Solidity does not clear nested mapping entries inside the deleted struct. Old delegatedSigners, approvedBeneficiaries, custom fee mappings, and zero-fee exemption mappings remain in storage.

When the same address is later re-added through `addBenefactor()`, the function only sets `isActive = true`. The old mapping entries therefore become live again. In particular, a previously accepted delegated signer can immediately submit swaps for the re-added benefactor and route output to a previously approved beneficiary without a fresh `setDelegatedSigner()` / `confirmDelegatedSigner()` flow or new beneficiary approval.

The scope documentation acknowledges that some old benefactor state can become active again on re-add, but the listed examples focus on epoch/period usage, nonces, and historical data. It does not make clear that executable authorization and fee/exemption mappings also revive.

RECOMMENDATION

If re-adding a benefactor is intended to preserve prior authorization/configuration state, update the documentation to explicitly state that old delegated signers, approved beneficiaries, custom fees, and zero-fee exemptions become active again on re-add, and require operators to review or revoke them before reactivation. If re-add is intended to behave like fresh onboarding, track benefactor onboarding generations separately from `isActive` and key authorization/config mappings by the current generation, or otherwise reset those mappings during re-add.

RESOLUTION

Status: Acknowledged.

LOW **L-05**

ACKNOWLEDGED

Aggregate Oracle Counts Duplicate Sources

CATEGORY	Oracle
LOCATION	AggregateOracleFeed.sol
REVIEW	Remediation Review

— DESCRIPTION

AggregateOracleFeed enforces quorum by counting oracle wrapper contracts, but it does not verify that those wrappers point to distinct underlying data sources. Different ChainlinkOracleFeed contracts can wrap the same Chainlink aggregator/proxy, and different PythOracleFeed contracts can wrap the same (oracle, priceId) pair.

As a result, minNumberOfOracles = 2 can be satisfied by two wrappers that both read from the same underlying source. A PSM collateral using this aggregate feed can pass oracle quorum and execute swaps with less source diversity than operators expect.

For example, two distinct ChainlinkOracleFeed wrappers can both be deployed with the same Chainlink aggregator address. AggregateOracleFeed accepts both because their wrapper addresses differ, getPrice() counts both successful reads, and the PSM then treats the aggregate feed as satisfying a two-oracle quorum even though only one underlying Chainlink source is used.

— RECOMMENDATION

Track and reject duplicate underlying oracle identities when adding feeds, such as the Chainlink aggregator/proxy address or the Pyth (oracle, priceId) pair. Alternatively, document that aggregate quorum counts adapter contracts only and that source uniqueness must be enforced operationally during oracle onboarding.

— RESOLUTION

Status: Acknowledged.

LOW **L-06**

ACKNOWLEDGED

PSM Ignores Pyth Confidence Bounds

CATEGORY	Oracle
LOCATION	PythOracleFeed.sol, PSM.sol
REVIEW	Remediation Review

— DESCRIPTION

PSM depeg checks use the Pyth center price after PythOracleFeed validates only that confidence / price $\leq$ maxConfidenceBps. A swap can pass even when the conservative price including confidence is outside the configured minOraclePrice or maxOraclePrice bound.

For example, with center price 1.00, confidence 1.5%, maxConfidenceBps = 200, minOraclePrice = 0.99e18, and maxOraclePrice = 1.01e18, the adapter returns 1.00e18. Both PSM directions can execute even though price - confidence = 0.985e18 is below the swapForAsset floor and price + confidence = 1.015e18 is above the swapForCollateral ceiling.

— RECOMMENDATION

Enforce an alignment rule between Pyth maxConfidenceBps and each collateral's PSM bounds, or use adverse-side confidence-adjusted prices for depeg checks. If center-price checks are intentional, document that PSM bounds are center-price bounds and must include enough confidence buffer.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-01**

ACKNOWLEDGED

PSM Deploy Config Values Can Wrap

CATEGORY	Informational
LOCATION	scripts/deploy/DeployPSM.s.sol
REVIEW	Remediation Review

— DESCRIPTION

DeployPSM reads numeric environment variables as uint256 but casts them to uint128 before validation. The affected values are MAX_SWAP_FOR_ASSET_PER_EPOCH, MAX_SWAP_FOR_COLLATERAL_PER_EPOCH, and PEG_PRICE. Because explicit Solidity downcasts do not revert, an oversized environment value can wrap to a smaller value before the PSM constructor receives it. This can deploy the PSM with unexpectedly tiny swap limits, or allow an oversized PEG_PRICE input to wrap into a valid-looking value before the constructor's peg-price bound is checked. As a result, a malformed deployment environment can silently produce materially different PSM configuration than the operator intended, instead of reverting during deployment.

— RECOMMENDATION

Validate deployment environment values as uint256 before downcasting. Ensure each value is $\leq \text{type(uint128).max}$, and validate PEG_PRICE against the PSM maximum before casting. Also compute derived period limits in uint256 and validate they fit in uint128 before assigning them to GlobalConfig.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-02

ACKNOWLEDGED

Stale Oracle Aggregation Docs

CATEGORY	Documentation
LOCATION	docs
REVIEW	Remediation Review

DESCRIPTION

The documentation still describes `AggregateOracleFeed` as using arithmetic mean / average aggregation. `docs/SCOPE.md` states that the protocol uses arithmetic mean rather than median, `docs/ARCHITECTURE.md` lists average price calculation as an aggregate oracle feature, and `docs/FORK_TESTING.md` says the fork test verifies the average of all four oracles.

This is stale. The current `AggregateOracleFeed` implementation and interface document median aggregation, and the code returns `Math.median(validPricesArray)`.

RECOMMENDATION

Update the documentation and comments to state that `AggregateOracleFeed` uses median aggregation, not arithmetic mean / average.

RESOLUTION

Status: Acknowledged.

INFORMATIONAL **I-03**

ACKNOWLEDGED

Inactive Benefactor Can Be Spent As Custodian

CATEGORY	Informational
LOCATION	PSM.sol
REVIEW	Remediation Review

— DESCRIPTION

PSM prevents a custodian address from being an active benefactor, but the check only looks at `benefactorState[address].config.isActive`. After a benefactor is disabled or removed, the same address can be configured as an asset or collateral send custodian while any old ERC20 approval to the PSM remains live.

Normal swaps by another active benefactor can then pull tokens from that inactive/offboarded benefactor through the send-custodian leg. This is possible when disabled or removed benefactors do not immediately revoke approvals, and it weakens the intended emergency/offboarding separation between benefactors and custodians. It requires privileged custodian configuration, so the impact is limited to a role-transition and allowance-reuse hazard rather than an unprivileged drain.

— RECOMMENDATION

Track benefactor registration/history separately from active status and reject current or formerly registered benefactors as send custodians unless an explicit admin migration/acknowledgement clears the relationship. Emergency offboarding runbooks should also require revoking PSM allowances from disabled or removed benefactors before any custodian reuse.

— RESOLUTION

Status: Acknowledged.

INFORMATIONAL I-04

ACKNOWLEDGED

Unsafe Downcast Before Quote Selection

CATEGORY	Math
LOCATION	https://github.com/GuardianOrg/onchain-minting-internal-team1-1782152411467/blob/354bc9b8019586b8e1d75fc8c8998a8507b55a1b/src/swap/PSM.sol#L1617-L1619 https://github.com/GuardianOrg/onchain-minting-internal-team1-1782152411467/blob/354bc9b8019586b8e1d75fc8c8998a8507b55a1b/src/minting/OnChainMinting.sol#L1127-L1132
REVIEW	Remediation Review

DESCRIPTION

OnChainMinting and PSM compute peg-based and oracle-based quote branches, cast each branch to uint128, and only then select the lower value. If the branch that should be discarded exceeds uint128, `SafeCast` reverts before the safe quote can be chosen.

For the intended stablecoin use case, such an extreme oracle value is unlikely under normal market conditions. The main concern is `_collateralConfig.oracleFeed` misconfiguration or a faulty oracle adapter, such as a wrong feed address, incorrect decimal scaling, or a custom feed returning extreme values. `AggregateOracleFeed` reduces this risk by applying min/max price filtering before returning a price.

RECOMMENDATION

Consider keeping the quote branches as uint256, select the lower value in uint256, and downcast only the final selected `amountOut` to uint128.

RESOLUTION

Status: Acknowledged.

§ 05

Disclaimer.

This report is not, nor should be considered, an "endorsement" or "disapproval" of any particular project or team. This report is not, nor should be considered, an indication of the economics or value of any "product" or "asset" created by any team or project that contracts Guardian to perform a security assessment. This report does not provide any warranty or guarantee regarding the absolute bug-free nature of the technology analyzed, nor do they provide any indication of the technologies proprietors, business, business model or legal compliance.

This report should not be used in any way to make decisions around investment or involvement with any particular project. This report in no way provides investment advice, nor should be leveraged as investment advice of any sort. This report represents an extensive assessing process intending to help our customers increase the quality of their code while reducing the high level of risk presented by cryptographic tokens and blockchain technology.

Blockchain technology and cryptographic assets present a high level of ongoing risk. Guardian's position is that each company and individual are responsible for their own due diligence and continuous security. Guardian's goal is to help reduce the attack vectors and the high level of variance associated with utilizing new and consistently changing technologies, and in no way claims any guarantee of security or functionality of the technology we agree to analyze.

The assessment services provided by Guardian is subject to dependencies and under continuing development. You agree that your access and/or use, including but not limited to any services, reports, and materials, will be at your sole risk on an as-is, where-is, and as-available basis.

Cryptographic tokens are emergent technologies and carry with them high levels of technical risk and uncertainty. The assessment reports could include false positives, false negatives, and other unpredictable results. The services may access, and depend upon, multiple layers of third-parties.

Notice that smart contracts deployed on the blockchain are not resistant from internal/external exploit. Notice that active smart contract owner privileges constitute an elevated impact to any smart contract's safety and security. Therefore, Guardian does not guarantee the explicit security of the audited smart contract, regardless of the verdict.

§ 06

About Guardian.

Guardian is a smart contract security firm. Every engagement is worked by two independent teams competing on the same scope, each with its own tooling and methods, so no single reviewer's blind spot survives.

\$45B+DIGITAL ASSETS UNDER
REVIEW**200+**

ENGAGEMENTS COMPLETED

500+CRITICAL & HIGH FINDINGS
REPORTED

Reach us at audits@guardianaudits.com.